

Counting ten-element matroids by aggregating extensions

ainta

Abstract

We give an exact algorithm for counting matroid isomorphism classes using catalogues of smaller matroids. For each parent and an automorphism, the algorithm counts invariant modular cuts by branching on lower flats and counting independent sets on the remaining hyperplanes. Integer moments recover the Burnside terms for permutations with a fixed point; basis-exchange formulas supply the remaining terms. A separate sparse-paving calculation allows the general extension worker to omit that family. We give pseudocode and correctness proofs for these reductions and their composition. The implementation returns 3 222 227 959 444 rank-five classes on ten elements and 3 232 000 741 644 classes across all ranks.

1 The problem and the algorithm

Let $u_{n,r}$ be the number of rank- r matroids on n elements up to isomorphism, and let $L_{n,r}$ be the number on the fixed labeled set $[n]$. All matroids are included: loops, parallel elements, coloops, and disconnected matroids are allowed. Isomorphism means a permutation of the ground set. Duality gives $u_{n,r} = u_{n,n-r}$, but distinct duals remain distinct classes.

The algorithm generates smaller parents and counts their extensions in aggregate. Its two main reductions are

one parent and one automorphism $\longrightarrow$ lower-flat branches $\longrightarrow$ independent-set counts,
parent extension counts $\longrightarrow$ integer moments $\longrightarrow$ unpointed isomorphism classes.

The first reduction avoids constructing the final matroids. The second avoids a separate isomorphism test for each possible extension.

For ten elements, rank five is the large middle-rank problem. The computation splits it into sparse-paving and non-sparse-paving matroids:

$$u_{10,5} = u_{10,5}^{\text{SP}} + \Delta_{10,5} = 2\,630\,337\,889\,305 + 591\,890\,070\,139. \quad (1)$$

The full algorithm also computes ranks three and four, uses elementary formulas for ranks zero through two, and obtains the remaining ranks by duality. Published counts are used for comparison, not as inputs.

The mathematical inputs are complete sets $\mathcal{P}_{q,r}$ of isomorphism-class representatives on q elements, with exact automorphism groups. The implementation generates the catalogues from the empty matroid using the established extension generator of [4, 6]; the extension worker computes their automorphism groups from the rank functions. The generator produces $|\mathcal{P}_{9,5}| = 190\,214$, agreeing with [2]. Modular-cut extension theory is due to Crapo [1]; canonical construction paths are standard [3]. The subject of this note is the aggregate counting algorithm built on these ingredients.

2 Counting extensions of one parent

Write ρ_N for the rank function of a matroid N on E , and set $r = \rho_N(E)$. A flat is a set equal to its closure; a hyperplane is a flat of rank $r - 1$. Two flats form a modular pair when

$$\rho_N(F) + \rho_N(H) = \rho_N(F \cup H) + \rho_N(F \cap H). \quad (2)$$

Let $e \notin E$ be the element to be added.

2.1 The Boolean formulation

A modular cut $\mathcal{C} \subseteq \mathcal{F}(N)$ is upward closed and closed under intersection of modular pairs. Crapo's correspondence [1] identifies it with a unique extension M satisfying $M \setminus e = N$. In this correspondence,

$$F \in \mathcal{C} \iff e \in \text{cl}_M(F). \quad (3)$$

In particular, for $X \subseteq E$,

$$\rho_M(X) = \rho_N(X), \quad \rho_M(X \cup \{e\}) = \rho_N(X) + 1 - \mathbf{1}_{\{\text{cl}_N(X) \in \mathcal{C}\}}. \quad (4)$$

The empty cut gives a coloop extension and raises the rank by one. Every nonempty cut contains E and gives a rank-preserving extension.

Introduce $x_F \in \{0, 1\}$ for every flat F . The nonempty cuts are exactly the satisfying assignments of

$$x_E = 1, \quad (5)$$

$$\neg x_F \vee x_H \quad (F \subsetneq H), \quad (6)$$

$$\neg x_F \vee \neg x_H \vee x_{F \cap H} \quad (F, H \text{ a modular pair}). \quad (7)$$

Thus counting extensions means counting these assignments, with the parent labels fixed.

For $g \in \text{Aut}(N)$, require the cut to be g -invariant by identifying flat variables in the same orbit under $\langle g \rangle$. Simplify repeated literals and delete tautological clauses. Denote the resulting formula by $\Phi_{N,g}$. Its orbit variables retain a well-defined flat rank.

Lemma 2.1. *The models of $\Phi_{N,g}$ are in bijection with rank-preserving extensions of N fixed by the permutation that acts as g on E and fixes e .*

Proof. An assignment to orbit variables is exactly a g -invariant assignment to flats. Equations (5)–(7) say precisely that its selected flats form a nonempty modular cut. The extension correspondence is equivariant: equation (4) is preserved by g exactly when the cut is preserved. Distinct cuts give distinct extensions. $\square$

2.2 Why the remaining choices form a graph

Call a flat *lower* if its rank is at most $r - 2$. Branch on lower-flat orbit variables and apply unit propagation after every assignment. Once all lower variables are assigned, only hyperplane variables can remain undecided.

Lemma 2.2 (Hyperplane reduction). *After all lower variables are assigned and unit propagation has reached a consistent fixed point, the remaining models of $\Phi_{N,g}$ are in bijection with the independent sets of a graph on the undecided hyperplane orbits. Two distinct orbit vertices are adjacent if they contain hyperplanes H, K with $\rho_N(H \cap K) = r - 2$.*

Proof. Distinct hyperplanes span N , so (2) becomes $\rho_N(H \cap K) = r - 2$. Their intersection is a lower flat. If its variable is true, upward closure forces both hyperplanes true. Consequently, whenever both hyperplane variables remain undecided, their modular-pair clause reduces to $\neg x_H \vee \neg x_K$.

Every other clause has at most one undecided variable: its other flats are lower or equal to E . Such a clause is either satisfied or has already been resolved by unit propagation. If two conflicting hyperplanes lie in the same orbit, the reduced clause is unary and that orbit cannot remain undecided. Thus all residual clauses forbid selecting the two ends of an edge, and no further constraints remain. Selecting an independent set therefore specifies exactly one completion of the cut. $\square$

For fixed (N, g) the conflict graph can be built once. Lower-flat branches change only its allowed vertex set. A nine-element rank-five parent has at most $\binom{9}{4} = 126$ hyperplanes and at most $\sum_{j=0}^3 \binom{9}{j} = 130$ lower flats: a rank- j flat is the closure of some j -element basis. This explains the graph size used by the worker.

2.3 The independent-set kernel

For a graph G and $S \subseteq V(G)$, let $i_G(S)$ count independent sets of $G[S]$, including the empty set. Write $N_G(v)$ for the open neighborhood of v . Algorithm 1 uses exact state keys in a cache belonging to a fixed graph.

Algorithm 1. $\text{IS}_G(S)$

1. If $S = \emptyset$, return 1. If S is cached, return its value.
 2. Save $S_0 \leftarrow S$. Remove all isolated vertices of $G[S]$; let k be their number and S the remaining set.
 3. If $S = \emptyset$, set $a \leftarrow 1$.
 4. Otherwise, if component splitting finds a component $C \subsetneq S$, set $a \leftarrow \text{IS}_G(C) \text{IS}_G(S \setminus C)$.
 5. Otherwise, if $G[S]$ is a path on t vertices, set $a \leftarrow F_{t+2}$; if it is a cycle, set $a \leftarrow F_{t-1} + F_{t+1}$.
 6. Otherwise choose $v \in S$ of maximum degree and set $a \leftarrow \text{IS}_G(S \setminus \{v\}) + \text{IS}_G(S \setminus (\{v\} \cup N_G(v)))$.
 7. Cache and return $2^k a$ under the key S_0 .
-

Here $F_0 = 0, F_1 = 1$ are the Fibonacci numbers. The implementation attempts component splitting when fewer than 45 vertices remain or the maximum degree is below 5; otherwise it branches directly.

Proposition 2.3. *Algorithm 1 returns $i_G(S)$.*

Proof. An isolated vertex can be independently included or excluded, giving 2^k . Choices on distinct components are independent, so their counts multiply. Partitioning by whether v is selected gives

$$i_G(S) = i_G(S \setminus \{v\}) + i_G(S \setminus (\{v\} \cup N_G(v))).$$

For paths, the same recurrence at an endpoint gives F_{t+2} ; conditioning on one vertex of a cycle gives $F_{t+1} + F_{t-1}$. Every recursive argument is smaller, proving termination and correctness by induction on $|S|$. Cache entries store previously established values of the same function. Eviction changes only the amount of repeated work. $\square$

2.4 The extension worker

We allow a set B of *marked* orbit variables and a flag b requiring at least one of them to be true. Ordinary extension counting uses $b = 0$. Section 4 specifies the marks for non-sparse-paving counting. Build $\Phi_{N,g}$ and its hyperplane graph once, and start with $x_E = 1$.

Algorithm 2. $\text{Cut}(\alpha, b)$ for fixed $\Phi_{N,g}$, G , and B

1. Propagate unit clauses from the partial assignment α . If a contradiction occurs, return 0.
 2. If a marked variable is true, set $b \leftarrow 0$.
 3. If an undecided lower variable v exists, return $\text{Cut}(\alpha \cup \{v = 1\}, b) + \text{Cut}(\alpha \cup \{v = 0\}, b)$, using a separate state for each branch.
 4. Let S be the set of undecided hyperplane variables.
 5. If $b = 0$, return $\text{IS}_G(S)$.
 6. Set $a \leftarrow 0$. List the marked vertices in S as $v_1, \dots, v_t$.
 7. For $j = 1, \dots, t$: set $S \leftarrow S \setminus \{v_j\}$ and $a \leftarrow a + \text{IS}_G(S \setminus N_G(v_j))$.
 8. Return a .
-

The C++ worker chooses a lower variable with maximum clause occurrence count and implements separate branch states by a reversible assignment trail. Its graph cache is fresh for each (N, g) ; a bare vertex-mask key cannot be reused across different graphs.

Proposition 2.4 (Worker correctness). *Algorithm 2 counts exactly the models extending α , subject to the requirement that some marked variable is true when $b = 1$.*

Proof. Unit propagation forces values shared by every satisfying completion. A contradiction gives no completions. Branching on v partitions all completions into two disjoint sets and decreases the number of undecided lower variables. At a leaf, Lemma 2.2 and Proposition 2.3 give the unrestricted count.

If a marked variable must still be selected, partition the independent sets by their first selected vertex in $v_1, \dots, v_t$. For the j th part, $v_1, \dots, v_{j-1}$ are excluded and v_j is included. The remaining choices are precisely the independent sets of $G[S_0 \setminus (\{v_1, \dots, v_j\} \cup N_G(v_j))]$, where S_0 was the initial leaf set. These are the terms in lines 6–7. They are disjoint and exhaust the required models; when $t = 0$ their sum is zero. $\square$

Example 2.5 (Five extensions of a three-point line). Take $N = U_{2,3}$. Its flats are $\emptyset$, the three singletons, and E . If $x_\emptyset = 1$, every flat is selected: the new element is a loop. If $x_\emptyset = 0$, the hyperplane graph is K_3 , with four independent sets. The empty set gives $U_{2,4}$, and each singleton makes e parallel to one existing point. Thus there are $1 + 4 = 5$ rank-preserving labeled extensions. For the automorphism (12), the orbit of the first two hyperplanes has an internal conflict and is excluded in the non-loop branch. Only the empty choice and the third hyperplane remain, giving $1 + 2 = 3$ invariant extensions. The coloop extension is separate and has rank three.

3 From parent counts to isomorphism classes

Let $F_{n,r}(\lambda)$ be the number of labeled rank- r matroids fixed by one permutation of cycle type $\lambda \vdash n$. If $a_j(\lambda)$ is the number of parts equal to j , set

$$z_\lambda = \prod_{j \geq 1} j^{a_j(\lambda)} a_j(\lambda)!$$

There are $n!/z_\lambda$ permutations of this type. Burnside's lemma gives

$$u_{n,r} = \sum_{\lambda \vdash n} \frac{F_{n,r}(\lambda)}{z_\lambda}. \quad (8)$$

The identity term $F_{n,r}(1^n)$ is $L_{n,r}$.

3.1 Permutations with a fixed element

Put $q = n - 1$. For $N \in \mathcal{P}_{q,r}$ and $g \in \text{Aut}(N)$, let $c(N, g)$ be the output of Algorithm 2 with no marked-variable requirement. For every $\mu \vdash q$, define the integer

$$\begin{aligned} T_\mu^{(r)} &= \sum_{N \in \mathcal{P}_{q,r}} \frac{q!}{|\text{Aut}(N)|} \sum_{\substack{g \in \text{Aut}(N) \\ \text{type}(g) = \mu}} c(N, g) \\ &+ \sum_{N \in \mathcal{P}_{q,r-1}} \frac{q!}{|\text{Aut}(N)|} \#\{g \in \text{Aut}(N) : \text{type}(g) = \mu\}. \end{aligned} \quad (9)$$

The second line counts coloop extensions. Write $\mu \cup (1)$ for adjoining one part of size one.

Theorem 3.1 (Parent-moment identity). *For every $\mu \vdash n - 1$,*

$$F_{n,r}(\mu \cup (1)) = \frac{z_\mu T_\mu^{(r)}}{(n-1)!}. \quad (10)$$

Proof. Fix the added label $e = n$. Count pairs (M, g) , where M is a labeled rank- r matroid on $[n]$, g is a permutation of $[q]$ of type μ , and g extended by fixing e preserves M .

Delete e . If e is not a coloop, the parent has rank r , is preserved by g , and Lemma 2.1 supplies $c(N, g)$ choices. If e is a coloop, the parent has rank $r - 1$ and there is exactly one extension. Each parent type has $q!/|\text{Aut}(N)|$ labelings. Summing by parent therefore gives precisely (9).

Alternatively, there are $q!/z_\mu$ choices of g . They are conjugate in S_q , so each fixes $F_{n,r}(\mu \cup (1))$ matroids. Equating the two counts proves (10). $\square$

In particular, the fixed-point part of (8) is

$$\sum_{\mu \vdash q} \frac{T_\mu^{(r)}}{q! (a_1(\mu) + 1)}, \quad (11)$$

because $z_{\mu \cup (1)} = (a_1(\mu) + 1)z_\mu$. This factor is the precise correction for adding a distinguished fixed element. Summing extension orbits directly would instead count pointed matroids (M, e) , with one occurrence per element orbit of M .

Reducing automorphism work. If $g' = hgh^{-1}$ for $h \in \text{Aut}(N)$, the map $\mathcal{C} \mapsto h(\mathcal{C})$ bijects the invariant cuts for g and g' . Thus each inner sum in (9) may be evaluated using one representative per conjugacy class of $\text{Aut}(N)$, multiplied by its class size. Equal cycle types in S_q alone do not establish this bijection for a fixed parent: the conjugating permutation must preserve N .

3.2 Permutations without a fixed element

For a representative σ of type $\lambda \vdash n$, introduce one Boolean variable y_O for every orbit O of r -subsets under $\langle \sigma \rangle$. Let $[B]$ denote the orbit of B . Use the formula

$$\Psi_{n,r,\sigma} = \left(\bigvee_O y_O \right) \wedge \bigwedge_{\substack{B,C \in \binom{[n]}{r} \\ a \in B \setminus C}} \left(\neg y_{[B]} \vee \neg y_{[C]} \vee \bigvee_{b \in C \setminus B} y_{[(B \setminus \{a\}) \cup \{b\}]} \right). \quad (12)$$

Write $\#MC(\Psi)$ for its exact number of satisfying assignments.

Proposition 3.2. $\#MC(\Psi_{n,r,\sigma}) = F_{n,r}(\lambda)$.

Proof. An assignment selects a unique σ -invariant family of r -subsets. The first clause makes it nonempty. The remaining clauses are exactly the basis-exchange axiom, which characterizes the basis families of rank- r matroids. Conversely, the bases of every fixed matroid give one satisfying assignment. No auxiliary variables occur, so the correspondence preserves the number of models without any projection factor. $\square$

For a hard formula we choose k variables and count all 2^k restrictions. Every model has exactly one assignment to these variables, hence

$$\#MC(\Psi) = \sum_{\eta \in \{0,1\}^k} \#MC(\Psi \wedge \eta). \quad (13)$$

These jobs can be scheduled independently. At $n = 10$ there are 42 cycle types: 30 with a fixed point and 12 without one. The latter are the only types sent to the full basis-formula counter.

Algorithm 3. Full rank count $\text{RankCount}(n, r)$, $1 \leq r < n$

1. Set $q \leftarrow n - 1$ and $T_\mu \leftarrow 0$ for every $\mu \vdash q$.
 2. For each $N \in \mathcal{P}_{q,r}$, put $w \leftarrow q! / |\text{Aut}(N)|$. For every conjugacy-class representative $g \in \text{Aut}(N)$ with class size s , compute $c \leftarrow \text{Cut}(x_E = 1, 0)$ and add wsc to $T_{\text{type}(g)}$.
 3. For each $N \in \mathcal{P}_{q,r-1}$ and each such class (s, g) , add $q!s / |\text{Aut}(N)|$ to $T_{\text{type}(g)}$.
 4. For each $\mu \vdash q$, set $F(\mu \cup (1)) \leftarrow z_\mu T_\mu / q!$, checking exact division.
 5. For each $\lambda \vdash n$ with no part 1, choose σ of that type and set $F(\lambda) \leftarrow \#MC(\Psi_{n,r,\sigma})$.
 6. Set $A \leftarrow \sum_{\lambda \vdash n} (n! / z_\lambda) F(\lambda)$. Return $(A/n!, F(1^n))$, checking exact division.
-

Theorem 3.3. *Given complete parent catalogues and their automorphism groups, Algorithm 3, with exact formula counts, returns $(u_{n,r}, L_{n,r})$.*

Proof. Proposition 2.4 supplies the correct invariant cut counts. Conjugacy-class aggregation gives the inner sums of (9). Theorem 3.1 supplies every fixed-point term, and Proposition 3.2 supplies every remaining term. These two sets of cycle types partition all of S_n . Equation (8) proves the unlabeled count; the identity fixes every labeled matroid and gives the second output. $\square$

4 Counting only the non-sparse-paving extensions

A rank- r matroid is paving if every circuit has size at least r , and sparse paving if both it and its dual are paving. For a rank-five matroid on ten elements this means that every circuit and every cocircuit has size at least five. Equivalently, it is paving and every hyperplane has at most five elements, since the complement of a hyperplane is a cocircuit.

In this section N has rank five on nine elements, and $M \setminus e = N$ also has rank five. A non-sparse-paving parent has no sparse-paving extension of this rank: deletion preserves its circuits, and a parent hyperplane of size at least six lies in a child hyperplane of size at least six.

Lemma 4.1 (The marked flats). *Suppose N is sparse paving and $\mathcal{C}$ is its nonempty extension cut. The extension M is non-sparse-paving if and only if $\mathcal{C}$ contains either a flat of rank at most three or a rank-four hyperplane of size five.*

Proof. If $F \in \mathcal{C}$ has rank at most three, choose a basis I of F . Then $e \in \text{cl}_M(I)$, so $I \cup \{e\}$ contains a circuit through e of size at most four. If a five-element hyperplane H belongs to $\mathcal{C}$, its closure in M is $H \cup \{e\}$, a six-element hyperplane. Its complement is a four-element cocircuit. Either case violates sparse paving.

Conversely, if M has a circuit D of size at most four, then $e \in D$ because N is paving. The flat $\text{cl}_N(D \setminus \{e\})$ has rank at most three and belongs to $\mathcal{C}$. Otherwise M is paving, so failure of sparse paving gives a hyperplane K with $|K| \geq 6$. It must contain e : a hyperplane of M avoiding e is a hyperplane of N , whose size is at most five. The flat $H = K \setminus \{e\}$ of N has at least five elements. Since every four-element subset of N is independent, $\rho_N(H) = 4$. Thus H is a hyperplane, $|H| = 5$, and $\rho_M(H \cup \{e\}) = \rho_N(H)$, proving $H \in \mathcal{C}$. $\square$

Let $c_{\text{bad}}(N, g)$ be the number of invariant nonempty cuts giving non-sparse-paving extensions. If N is not sparse paving, use Algorithm 2 with $b = 0$. Otherwise mark the orbit variables containing the flats in Lemma 4.1, and use $b = 1$. These marks are invariant under $\text{Aut}(N)$, so the conjugacy-class reduction still applies. Proposition 2.4 proves that the omitted branch is exactly the sparse-paving family.

4.1 The coloop contribution at the middle rank

Every rank-five coloop extension on ten elements is non-sparse-paving, because a coloop is a one-element cocircuit. Duality bijects $\mathcal{P}_{9,5}$ and $\mathcal{P}_{9,4}$ and preserves their automorphism groups as permutation groups. Consequently the two parent sums can be paired:

$$T_{\mu}^{\text{bad}} = \sum_{N \in \mathcal{P}_{9,5}} \frac{9!}{|\text{Aut}(N)|} \sum_{\substack{g \in \text{Aut}(N) \\ \text{type}(g) = \mu}} (c_{\text{bad}}(N, g) + 1), \quad \mu \vdash 9. \quad (14)$$

The $+1$ is the coloop extension of the dual parent. This pairing is used only at $(n, r) = (10, 5)$; the general rank calculation keeps the two parent sets in (9) separate.

Let $F_{10,5}^{\text{SP}}(\lambda)$ be the sparse-paving fixed count. Applying the proof of Theorem 3.1 to the non-sparse-paving family gives

$$F_{10,5}^{\text{bad}}(\lambda) = \begin{cases} z_{\mu} T_{\mu}^{\text{bad}} / 9!, & \lambda = \mu \cup (1), \\ \# \text{MC}(\Psi_{10,5,\sigma}) - F_{10,5}^{\text{SP}}(\lambda), & 1 \notin \lambda. \end{cases} \quad (15)$$

Thus

$$\Delta_{10,5} = \sum_{\lambda \vdash 10} \frac{F_{10,5}^{\text{bad}}(\lambda)}{z_\lambda}. \quad (16)$$

All fixed-point terms are obtained without visiting the all-marks-false extension family. The twelve derangement terms count all matroids by the basis formula and subtract the sparse-paving terms.

5 The sparse-paving calculation

For $1 \leq r < n$, let $J(n, r)$ be the Johnson graph: its vertices are the r -subsets of $[n]$, with adjacency when their intersection has size $r - 1$. Circuit-hyperplane families of rank- r sparse-paving matroids are exactly its independent sets [2]. Write $I(n, r) = i(J(n, r))$ for their labeled count. All graph orbits below are under ground-set permutations. When $n = 2r$, complementation also preserves $J(n, r)$; we do not quotient by it, because it corresponds to matroid duality.

5.1 The identity permutation

Distinguish $e = n$ and split a circuit-hyperplane family into A , the blocks containing e with e removed, and B , the blocks avoiding e . Define

$$\Gamma^+(A) = \left\{ b \in \binom{[n-1]}{r} : \exists a \in A, a \subset b \right\}.$$

Then A is independent in $J(n-1, r-1)$, B is independent in $J(n-1, r)$, and cross-adjacency is excluded exactly by $B \cap \Gamma^+(A) = \emptyset$.

Proposition 5.1 (Parent-orbit split). *If $\mathcal{A}$ contains one representative from every S_{n-1} -orbit of independent sets of $J(n-1, r-1)$, then*

$$I(n, r) = \sum_{A \in \mathcal{A}} \frac{(n-1)!}{|\text{Aut}(A)|} i(J(n-1, r) - \Gamma^+(A)). \quad (17)$$

Proof. Every circuit-hyperplane family has a unique pair (A, B) . For a fixed labeled A , its possible B are the independent sets of the displayed residual graph. Relabeling A gives an isomorphic residual graph, so all $(n-1)!/|\text{Aut}(A)|$ members of its orbit have the same number of extensions. Summing over the parent orbits counts every labeled family once. $\square$

5.2 Small parents by completion counts

For $(n, r) = (10, 5)$, complementation on [9] identifies $J(9, 5)$ with $G = J(9, 4)$. A parent A with k blocks forbids a set D that is the union of k disjoint five-vertex cliques: one clique consists of the five supersets of a fixed four-subset before complementation. Two distinct parent blocks cannot share a five-element superset, because A is independent in $J(9, 4)$.

For an independent set T of G , define $E(T) = \#\{H \subseteq V(G) : H \text{ independent, } T \subseteq H\}$. Let $o(H) = 9!/|\text{Aut}(H)|$ and $m_T(H) = \#\{T' \subseteq H : T' \cong T\}$, with isomorphism under S_9 .

Proposition 5.2 (Completion table and exact truncation). *Given a complete S_9 -orbit catalogue $\mathcal{H}$ of independent sets of G , we have*

$$E(T) = \frac{\sum_{H \in \mathcal{H}} o(H) m_T(H)}{o(T)}, \quad (18)$$

$$i(G - D) = \sum_{\substack{T \subseteq D \\ T \text{ independent}}} (-1)^{|T|} E(T). \quad (19)$$

If D is covered by k cliques, all terms with $|T| > k$ vanish.

Proof. For (18), count pairs (H, T') over all labeled independent sets H , with $T' \subseteq H$ and $T' \cong T$. Summing first over H gives the numerator. Each of the $o(T)$ copies of T has the same number $E(T)$ of completions, giving the denominator.

For (19), a fixed independent set H contributes $\sum_{T \subseteq H \cap D} (-1)^{|T|}$, which equals one when $H \cap D = \emptyset$ and zero otherwise. Every such T is independent. An independent set can meet each covering clique in at most one vertex, so its size is at most k . $\square$

The implementation tabulates $E(T)$ for $|T| \leq 5$ from the freshly generated nine-element sparse-paving catalogue. This handles all 251 parent types with $|A| \leq 5$. Larger parents use Algorithm 1 on the residual graph. This is a choice between two exact formulas for the same count.

5.3 The other permutations

For $\sigma \in S_n$, a fixed circuit-hyperplane family is a union of $\langle \sigma \rangle$ -orbits of r -subsets. Delete any orbit containing an internal Johnson edge. Form a graph Q_σ on the remaining orbits, joining two when an edge connects their members. Selecting independent orbit vertices is equivalent to selecting a σ -invariant independent set of $J(n, r)$; hence

$$F_{n,r}^{\text{SP}}(\text{type}(\sigma)) = i(Q_\sigma). \quad (20)$$

Algorithm 4. Sparse-paving fixed counts at $(n, r) = (10, 5)$

1. Generate the complete catalogue $\mathcal{H}$ of independent sets of $J(9, 4)$ modulo S_9 , including their automorphism orders.
 2. Compute $E(T)$ for the types with $|T| \leq 5$ using (18).
 3. For each $A \in \mathcal{H}$, compute its residual count by (19) if $|A| \leq 5$, and by IS otherwise.
 4. Set $F^{\text{SP}}(1^{10})$ to the orbit-weighted sum (17).
 5. For every other $\lambda \vdash 10$, choose a permutation σ of type λ , construct Q_σ , and set $F^{\text{SP}}(\lambda) \leftarrow \text{IS}_{Q_\sigma}(V(Q_\sigma))$.
 6. Return the table F^{SP} and $u_{10,5}^{\text{SP}} = \sum_{\lambda \vdash 10} F^{\text{SP}}(\lambda) / z_\lambda$.
-

Corollary 5.3. *Algorithm 4 returns the sparse-paving fixed counts and their unlabeled total.*

Proof. Propositions 5.1 and 5.2 give the identity term. The orbit-selection bijection proves (20) for every other term. Proposition 2.3 evaluates these graph counts, and Burnside's lemma gives the final sum. $\square$

6 The complete ten-element algorithm

For the three smallest ranks, direct formulas are sufficient. For $n \geq 2$,

$$\begin{aligned} (u_{n,0}, L_{n,0}) &= (1, 1), \\ (u_{n,1}, L_{n,1}) &= (n, 2^n - 1), \\ u_{n,2} &= \sum_{s=2}^n (p(s) - 1), \quad L_{n,2} = B_{n+1} - 2^n. \end{aligned} \tag{21}$$

Here $p(s)$ is the integer partition number and B_s is the Bell number. Rank one is determined by its nonempty set of nonloops. A rank-two matroid is determined by its loops and a partition of the nonloops into at least two parallel classes. Up to isomorphism, the class sizes are an integer partition of some $s \leq n$ with at least two parts. For labeled counting, adjoin a marker to the loop block and partition the resulting $n + 1$ elements. The forbidden cases with zero or one nonloop block number $1 + (2^n - 1) = 2^n$. This proves (21).

Algorithm 5. All matroids on ten elements

1. Generate the parent catalogues $\mathcal{P}_{9,r}$ needed below, with their automorphism groups, and the sparse-paving catalogue for Algorithm 4.
 2. Run Algorithm 4 to obtain $F^{\text{SP}}(\lambda)$ for all $\lambda \vdash 10$.
 3. Set $T_\mu^{\text{bad}} \leftarrow 0$ for all $\mu \vdash 9$. For each $N \in \mathcal{P}_{9,5}$ and each automorphism conjugacy class (s, g) , compute $c_{\text{bad}}(N, g)$ as in Section 4 and add $9!s(c_{\text{bad}}(N, g) + 1)/|\text{Aut}(N)|$ to $T_{\text{type}(g)}^{\text{bad}}$.
 4. Obtain $F^{\text{bad}}(\lambda)$ from (15), and set $F_5(\lambda) \leftarrow F^{\text{SP}}(\lambda) + F^{\text{bad}}(\lambda)$. Check the exact divisions and nonnegativity.
 5. Set $u_{10,5} \leftarrow \sum_{\lambda \vdash 10} F_5(\lambda)/z_\lambda$ by integer Burnside accumulation, and $L_{10,5} \leftarrow F_5(1^{10})$.
 6. Compute ranks 3 and 4 with Algorithm 3 and ranks 0, 1, 2 with (21).
 7. Return $(u_{10,5} + 2 \sum_{r=0}^4 u_{10,r}, L_{10,5} + 2 \sum_{r=0}^4 L_{10,r})$.
-

Theorem 6.1 (Correctness of the combined count). *With complete parent catalogues, their exact automorphism groups, and exact counts of the basis formulas, Algorithm 5 returns the numbers of unlabeled and labeled matroids on ten elements, across all ranks.*

Proof. Corollary 5.3 gives the fixed counts for the sparse-paving family. Lemma 4.1 and Proposition 2.4 give the invariant non-sparse-paving extensions of every rank-five parent. Equation (14) also includes every coloop extension through the dual rank-four parent. Theorem 3.1, restricted to this isomorphism-invariant family, therefore gives every non-sparse-paving fixed-point term. Proposition 3.2 gives the full derangement terms, from which the sparse-paving terms are subtracted.

The two families are disjoint and exhaustive, so their fixed counts add to $F_{10,5}(\lambda)$ for every cycle type. Burnside's lemma and the identity term give the two rank-five counts. Theorem 3.3 and (21) establish ranks zero through four. Finally, duality is a bijection on both labeled matroids and isomorphism classes, pairing each rank below five with a distinct rank above five. The last line sums these disjoint ranks exactly once. $\square$

The algorithm's substantial savings occur before the final Burnside sum. It counts many hyperplane selections with a single graph call, avoids the large sparse-paving extension family in

the general worker, and replaces the hardest small-parent graph counts by a completion table. The moments make these aggregate counts usable without constructing the ten-element objects to determine their automorphism groups.

7 Implementation and computation

7.1 Implementation

To compute $\text{Aut}(N)$, the worker backtracks over bijections of the parent ground set. A candidate image must have the same counts of containing subsets of each size and rank. After each assignment, it checks rank preservation on every subset of the assigned domain. Every automorphism passes both tests, while every completed bijection preserves the full rank function; thus the returned group is exact. Its conjugacy classes are formed by explicitly collecting hgh^{-1} for $h \in \text{Aut}(N)$.

The extension worker stores graph states as bit masks. Its implication propagation and reversible assignment trail implement Algorithm 2; the independent-set kernel implements Algorithm 1. Each parent and automorphism class defines an independent counting problem. Persistent processes take these jobs from a shared queue, retain the graph cache within a job, and return integer moment contributions. Moment accumulation and final reductions use arbitrary-precision integers.

The derangement formulas use one variable per basis orbit and no auxiliary variables. Ganak [7] is run in exact mode. For the type 2^5 at ranks four and five, the driver fixes eight variables of highest clause occurrence and sums all 256 resulting cubes, using (13). Completed parent rows and formula counts are checkpointed; a timed-out job has no numeric contribution until it completes.

The source correspondence is:

Part of the algorithm	Source file in the repository
Independent-set kernel	<code>src/split_count.cpp</code>
Invariant modular cuts	<code>src/extension_worker.cpp</code>
Small-parent completion table	<code>src/small_extensions.hpp</code>
Sparse-paving fixed counts	<code>scripts/count_sparse.py</code>
Basis-exchange formulas	<code>scripts/export_symmetry.py</code>
Moment reduction and all-rank sum	<code>scripts/count.py</code>

7.2 Results

Table 1 gives the outputs by rank. The full sums are

$$\boxed{u_{10} = 3\,232\,000\,741\,644}, \quad \boxed{L_{10} = 11\,727\,995\,799\,397\,397\,461}. \quad (22)$$

Ranks two through four agree with the published values in [5, Appendix A], which attributes the rank-four enumeration to [4].

The rank-five calculation separates as follows. In the middle row, “nonidentity numerator” means $\sum_{\lambda \neq 1^{10}} (10!/z_\lambda) F(\lambda)$; adding the first two rows and dividing by $10!$ gives the last row.

	Sparse paving	Non-sparse-paving
Identity fixed count	9 544 867 699 823 218 639	2 147 694 944 092 690 682
Nonidentity numerator	102 432 886 765 361	155 742 427 712 518
Unlabeled count	2 630 337 889 305	591 890 070 139

Table 1: Computed ten-element counts. Ranks $10 - r$ have the same values.

Rank r	Unlabeled $u_{10,r}$	Labeled $L_{10,r}$
0	1	1
1	10	1 023
2	128	677 546
3	10 037	16 869 404 175
4	4 886 380 924	17 716 560 870 661 325
5	3 222 227 959 444	11 692 562 643 915 909 321

The complete tables of fixed counts are recorded in `results/fixed_terms.json`. Appendix A lists the twelve rank-five derangement counts.

The recorded full run took 389.16 seconds on 64 physical cores of two AMD EPYC 9354 processors. This includes generating the parents from the empty matroid (3.83 seconds), the sparse-paving calculation (124.85 seconds), ranks three and four (6.92 and 76.72 seconds), and the remaining rank-five work (176.47 seconds). Compilation and the separate validation runs are excluded. The C++ programs were compiled with GCC 13.3.0 and `-O3 -march=native -std=c++17`.

7.3 Checks and reproduction

The generated catalogues reproduce all 55 size-rank entries through nine elements in [2], including 383 172 classes on nine elements. Across sizes zero through nine, all 385 370 canonical rank arrays match the reference catalogue [8]. The rank axioms, absence of isomorphic duplicates, and closure under duality were checked directly. For the 190 214 rank-five parents on nine elements, a separate automorphism-order check agrees with the worker; relabeling the parents also preserves every moment vector.

The aggregate algorithm itself reproduces the known values

(n, r)	$(6, 3)$	$(7, 3)$	$(8, 3)$	$(8, 4)$	$(9, 3)$	$(9, 4)$
$u_{n,r}$	38	108	325	940	1 275	190 214.

These cases exercise both the parent-moment terms and the derangement formulas. A fresh ten-element run with regenerated parents reproduces all 126 fixed counts for ranks three, four, and five. The result and check summaries are in `results/counts.json`, `results/validation.json`, and `results/provenance.json`.

The implementation is in the `matroid-count` repository [9], revision `d1aaa1468a01`. From its root, on Linux with Python 3.10+, GCC, Make, and Boost headers, run:

```
python3 scripts/setup.py --audit
make -j4
make audit
make test
python3 scripts/count.py --workdir runs/n10 --jobs 64
python3 scripts/validate.py --parents runs/n10/parents \
    --ten-run runs/n10 --out runs/n10/validation --jobs 8
```

The computed total is written to `runs/n10/total.json`. The generated parent ledgers, formulas, and solver logs remain in that run directory. The source tree contains the compact result summaries.

A Rank-five derangement terms

Each entry in Table 2 is the number of labeled matroids fixed by one permutation of the indicated type. The non-sparse-paving column is obtained by subtraction. Its contribution to the unlabeled count is the sum of that column divided, row by row, by z_λ .

Table 2: All twelve derangement types in S_{10} at rank five.

Cycle type	All matroids	Sparse paving	Non-sparse-paving
2^5	584 746 603	217 942 377	366 804 226
$4, 2^3$	792 347	34 257	758 090
$3^2, 2^2$	104 620	16 542	88 078
$6, 2^2$	5 620	1 146	4 474
$5, 3, 2$	129	4	125
$4^2, 2$	150 791	53 705	97 086
$8, 2$	637	389	248
$4, 3^2$	1 022	16	1 006
$7, 3$	28	1	27
$6, 4$	746	132	614
5^2	24 566	21 969	2 597
10	78	67	11

References

- [1] H. H. Crapo, Single-element extensions of matroids, *J. Res. Nat. Bur. Standards Sect. B* **69B** (1965), 55–65. doi:10.6028/jres.069B.003.
- [2] D. Mayhew and G. F. Royle, Matroids with nine elements, *J. Combin. Theory Ser. B* **98** (2008), 415–431. doi:10.1016/j.jctb.2007.07.005.
- [3] B. D. McKay, Isomorph-free exhaustive generation, *J. Algorithms* **26** (1998), 306–324. <https://users.cecs.anu.edu.au/~bdm/papers/orderly.pdf>.
- [4] Y. Matsumoto, S. Moriyama, H. Imai, and D. Bremner, Matroid enumeration for incidence geometry, *Discrete Comput. Geom.* **47** (2012), 17–43. doi:10.1007/s00454-011-9388-y.
- [5] M. Joswig and B. Schröter, Matroids from hypersimplex splits, *J. Combin. Theory Ser. A* **151** (2017), 254–284. doi:10.1016/j.jcta.2017.05.001.
- [6] IC matroid generator, source repository. <https://github.com/gmou3/matroid-generator>. Version used: 60645ae024e831fba3ebd35b2388515b0304140a.
- [7] Ganak exact model counter, version 2.6.4. <https://github.com/meelgroup/ganak>.
- [8] *Matroids with 9 elements*, reference catalogue, Zenodo. <https://zenodo.org/records/6825419>.
- [9] ainta, *Counting all matroids on ten elements*, source repository. <https://github.com/ainta/matroid-count>. Version used: d1aaa1468a01d983b14651a58874948dd357c606.